

QUESTIONS 8 / 8 completed

1-10 T

1-10 T

Write a Java program to create a simple thread that prints the numbers from 1 to 10.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {  
2     public static void main(String[] args) {  
3         Thread t = new Thread(() -> {  
4             for (int i = 1; i <= 10; i++) {  
5                 System.out.println(i);  
6             }  
7         });  
8         t.start();  
9     }  
10 }
```

OUTPUT

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

INPUT

Your INPUT goes here!

QUESTIONS 8 / 8 completed

Counter-T

COUNTER-T

Write a Java program to create a thread-safe counter that can be accessed by multiple threads simultaneously.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.concurrent.atomic.AtomicInteger;
2 public class Main {
3     public static void main(String[] args) throws InterruptedException {
4         Counter counter = new Counter();
5         Worker t1 = new Worker(counter);
6         Worker t2 = new Worker(counter);
7         Worker t3 = new Worker(counter);
8         t1.start();
9         t2.start();
10        t3.start();
11        t1.join();
12        t2.join();
13        t3.join();
14        System.out.println("Final Counter Value: " + counter.getCount());
15    }
16 }
17 class Counter {
18     private AtomicInteger count = new AtomicInteger(0);
19     public void increment() {
20         count.incrementAndGet();
21     }
```

OUTPUT

```
Final Counter Value: 3000
```

INPUT

Your INPUT goes here!

QUESTIONS 8 / 8 completed

M-Thread ▾

M-THREAD

Write a Java program to create multiple threads that perform some time-consuming task, such as calculating prime numbers or sorting an array, and compare their execution times.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Main {
2     static boolean isPrime(int n) {
3         if (n < 2) return false;
4         for (int i = 2; i * i <= n; i++) {
5             if (n % i == 0) return false;
6         }
7         return true;
8     }
9     static class PrimeThread extends Thread {
10        int start, end;
11        int count = 0;
12        PrimeThread(int start, int end) {
13            this.start = start;
14            this.end = end;
15        }
16        public void run() {
17            for (int i = start; i <= end; i++) {
18                if (isPrime(i)) count++;
19            }
20        }
21    }
```

OUTPUT

```
Single: primes=17984, time=31 ms
Multi : primes=17984, time=16 ms
```

INPUT

Your INPUT goes
here!

QUESTIONS 8 / 8 completed

Pro-Consume ▾

PRO-CONSUME

Write a Java program to create a producer-consumer scenario using multiple threads. The producer thread should add items to a queue, and the consumer thread should remove items from the queue and process them.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class ProducerConsumer {
2     static class Buffer {
3         private final java.util.Queue<Integer> queue = new java.util.LinkedList<
4         private final int capacity;
5         public Buffer(int capacity) {
6             this.capacity = capacity;
7         }
8         public synchronized void produce(int item) throws InterruptedException {
9             while (queue.size() == capacity) {
10                 wait();
11             }
12             queue.add(item);
13             System.out.println("Produced: " + item);
14             notifyAll();
15         }
16         public synchronized int consume() throws InterruptedException {
17             while (queue.isEmpty()) {
18                 wait();
19             }
20             int item = queue.poll();
21             System.out.println("Consumed: " + item);
```

OUTPUT

```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 6
```

INPUT

Your INPUT goes
here!

QUESTIONS 8 / 8 completed

Deadlock ▾

DEADLOCK

Write a Java program to create a deadlock scenario using multiple threads.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class DeadlockDemo {
2     public static void main(String[] args) {
3         final Object lock1 = new Object();
4         final Object lock2 = new Object();
5         Thread t1 = new Thread(() -> {
6             synchronized (lock1) {
7                 System.out.println("Thread 1: Holding lock1...");
8                 try {
9                     Thread.sleep(100);
10                } catch (InterruptedException e) {
11                    e.printStackTrace();
12                }
13                System.out.println("Thread 1: Waiting for lock2...");
14                synchronized (lock2) {
15                    System.out.println("Thread 1: Acquired lock2!");
16                }
17            }
18        });
19        Thread t2 = new Thread(() -> {
20            synchronized (lock2) {
21                System.out.println("Thread 2: Holding lock2...");
```

OUTPUT

```
Error: Execution timeout (>5 seconds) - possible infinite loop
```

INPUT

Your INPUT goes here!

QUESTIONS 8 / 8 completed

Wait-Notify

WAIT-NOTIFY

Write a Java program to demonstrate how to use the "wait" and "notify" methods to implement a producer-consumer scenario.

PROGRAM EDITOR

Java

Run

Save

```
1 public class WaitNotifyDemo {
2     static class Buffer {
3         private final int capacity;
4         private int itemCount = 0;
5         Buffer(int capacity) {
6             this.capacity = capacity;
7         }
8         public synchronized void produce(int value) throws InterruptedException {
9             while (itemCount == capacity) {
10                 System.out.println("Buffer full. Producer waits...");
11                 wait();
12             }
13             itemCount++;
14             System.out.println("Produced: " + value + " | Items in buffer: " + itemCount);
15             notify();
16         }
17         public synchronized void consume() throws InterruptedException {
18             while (itemCount == 0) {
19                 System.out.println("Buffer empty. Consumer waits...");
20                 wait();
21             }
22         }
23     }
24 }
```

OUTPUT

```
Produced: 1 | Items in buffer: 1
Consumed one item | Items in buffer: 0
Produced: 2 | Items in buffer: 1
Consumed one item | Items in buffer: 0
Produced: 3 | Items in buffer: 1
Consumed one item | Items in buffer: 0
Produced: 4 | Items in buffer: 1
Produced: 5 | Items in buffer: 2
Consumed one item | Items in buffer: 1
Produced: 6 | Items in buffer: 2
```

INPUT

Your INPUT goes here!

QUESTIONS 8 / 8 completed

Pool ▾

POOL

Write a Java program to create a thread pool that can execute multiple threads concurrently.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 import java.util.concurrent.ExecutorService;
2 import java.util.concurrent.Executors;
3 public class Main {
4     static class Task implements Runnable {
5         private String taskName;
6         public Task(String name) {
7             this.taskName = name;
8         }
9         @Override
10        public void run() {
11            System.out.println("Executing: " + taskName + " by " + Thread.currentThread().getName());
12            try {
13                Thread.sleep(1000);
14            } catch (InterruptedException e) {
15                System.out.println(taskName + " was interrupted.");
16            }
17            System.out.println("Completed: " + taskName);
18        }
19    }
20    public static void main(String[] args) {
21        ExecutorService executor = Executors.newFixedThreadPool(3);
```

OUTPUT

```
All tasks submitted to the thread pool.
Executing: Task-2 by pool-1-thread-2
Executing: Task-1 by pool-1-thread-1
Executing: Task-3 by pool-1-thread-3
Completed: Task-1
Completed: Task-2
Completed: Task-3
Executing: Task-5 by pool-1-thread-2
Executing: Task-4 by pool-1-thread-1
Completed: Task-5
```

INPUT

Your INPUT goes here!

QUESTIONS 8 / 8 completed

Join

JOIN

Write a Java program to demonstrate how to use the "join" method to wait for a thread to complete its execution before proceeding with the main thread.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     static class MyThread extends Thread {
3         public void run() {
4             try {
5                 System.out.println("Child thread is starting...");
6                 Thread.sleep(2000);
7                 System.out.println("Child thread has finished its work.");
8             } catch (InterruptedException e) {
9                 System.out.println("Child thread was interrupted.");
10            }
11        }
12    }
13    public static void main(String[] args) {
14        MyThread t1 = new MyThread();
15        System.out.println("Main thread is starting.");
16        t1.start();
17        try {
18            System.out.println("Main thread is now waiting for the child thread");
19            t1.join();
20        } catch (InterruptedException e) {
21            System.out.println("Main thread was interrupted.");
22        }
23    }
24 }
```

OUTPUT

```
Main thread is starting.
Main thread is now waiting for the child thread to finish...
Child thread is starting...
Child thread has finished its work.
Main thread is proceeding now that the child thread is done.
```

INPUT

Your INPUT goes here!

QUESTIONS 2 / 2 completed

Sahred

SAHRED

Write a Java program to demonstrate how to use synchronization to prevent multiple threads from accessing a shared resource simultaneously.

PROGRAM EDITOR

Java

✓ Updated

Run

Save

```
1 public class Main {
2     public static void main(String[] args) throws InterruptedException {
3         Counter counter = new Counter();
4         Worker t1 = new Worker(counter);
5         Worker t2 = new Worker(counter);
6         t1.start();
7         t2.start();
8         t1.join();
9         t2.join();
10        System.out.println("Final count value: " + counter.getCount());
11    }
12 }
13 class Counter {
14     private int count = 0;
15     public synchronized void increment() {
16         count++;
17     }
18     public int getCount() {
19         return count;
20     }
21 }
```

OUTPUT

```
Final count value: 2000
```

INPUT

Your INPUT goes here!

QUESTIONS 2 / 2 completed

AtomicInteger ▾

ATOMICINTEGER

Write a Java program to demonstrate how to use the AtomicInteger class to provide atomic access to a shared integer variable.

Output:
Shared value: 20000

This confirms that the two threads were able to increment the shared value atomically and the final value of the shared value is as expected. The exact number may vary as it

PROGRAM EDITOR

Java ▾

Run

Save

```
1 import java.util.concurrent.atomic.AtomicInteger;
2 public class AtomicIntegerDemo{
3     private static AtomicInteger sharedValue = new AtomicInteger(0);
4     public static void main(String[] args) throws InterruptedException {
5         Runnable incrementTask = () -> {
6             for (int i = 0; i < 10000; i++) {
7                 sharedValue.incrementAndGet();
8             }
9         };
10        Thread thread1 = new Thread(incrementTask);
11        Thread thread2 = new Thread(incrementTask);
12        thread1.start();
13        thread2.start();
14        thread1.join();
15        thread2.join();
16        System.out.println("Shared value: " + sharedValue.get());
17    }
18 }
```

OUTPUT

```
Shared value: 20000
```

INPUT

Your INPUT goes here!